Aprendizaje por Refuerzo

Métodos de Diferencias Temporales

Luis Daniel Hernández Molinero
Ciencia de la Computación e Inteligencia Artificial
Dept. Ingeniería de la Información y las Comunicaciones

Universidad de Murcia

Máster en Inteligencia Artificial

2025 / 2026

Índice de Contenidos

1. Introducción

2. Cálculo Incremental

3. Predicción
   3.1. Diferencias Temporales a un Paso: TD(0)
   3.2. Optimalidad de TD(0) - Por lotes

4. Control
   4.1. SARSA
   4.2. Q-Learning
   4.3. Sesgo de Maximización
        Regularización de Valores
        Promedio de Valores
            (Expected SARSA y Aprendizaje doble)

5. Referencias

Desarrollo

1. Introducción

2. Cálculo Incremental

3. Predicción
   3.1. Diferencias Temporales a un Paso: TD(0)
   3.2. Optimalidad de TD(0) - Por lotes

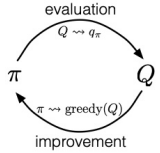
4. Control
   4.1. SARSA
   4.2. Q-Learning
   4.3. Sesgo de Maximización
        Regularización de Valores
        Promedio de Valores
            (Expected SARSA y Aprendizaje doble)

5. Referencias

Introducción

- El **aprendizaje TD es una combinación** de ideas de Monte Carlo y programación dinámica (DP).
- Al igual que los métodos de **Monte Carlo**, los métodos de Diferencias Temporales **aprenden directamente de la experiencia** en bruto sin un modelo de las dinámicas del entorno.
  - **Experiencia:** $S_0, A_0, R_1, S_1, A_1, \ldots, R_T$
  - **Se desconoce** $p(s', r|s, a)$

    No podemos usar $\pi'(s) = \arg\max_a \sum_{s',r} p(s', r|s, a)[r + \gamma V(s')]$, **usaremos** $\pi'(s) = \arg\max_a Q(s, a)$.
- Al igual que la **Programación Dinámica**, los métodos TD hacen **bootstrap**: actualizan estimaciones basadas en otras estimaciones aprendidas, sin esperar un resultado final.
  - **Utilizan bootstrapping**: $Q(S_t, A_t) \leftarrow \cdots\cdots \quad Q(S_{t+1}, A_{t+1}) \quad \cdots\cdots$
- Seguiremos la **Iteración de Politica Generalizada**, pero **no necesitamos el episodio entero** para evaluar.



Desarrollo

1. Introducción

2. Cálculo Incremental

3. Predicción
   3.1. Diferencias Temporales a un Paso: TD(0)
   3.2. Optimalidad de TD(0) - Por lotes

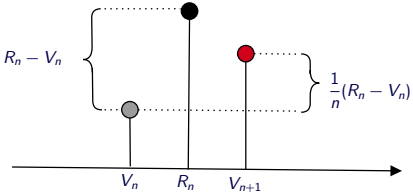
4. Control
   4.1. SARSA
   4.2. Q-Learning
   4.3. Sesgo de Maximización
        Regularización de Valores
        Promedio de Valores
            (Expected SARSA y Aprendizaje doble)

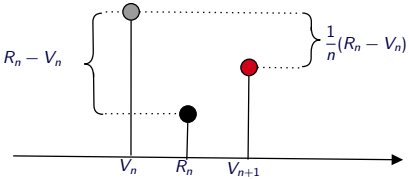
5. Referencias

Interpretación del Cálculo Incremental

- El **cálculo de un promedio** de forma **incremental** como: $V_{n+1} = \frac{1}{n}\sum_{i=1}^{n} R_i = V_n + \frac{1}{n}(R_n - V_n)$

- **Gráficamente:**





- **De forma general**, se puede ver de la siguiente manera:

$$NuevoEstimado = ViejoEstimado + TamañodelPaso \cdot (Objetivo - ViejoEstimado)\,.$$

  - Para obtener una nueva estimación para un objetivo, usamos la **estimación antigua**.

    Error de la estimación: $(Objetivo - ViejoEstimado)$
  - También usamos un **tamaño de paso**.

    El tamaño de paso establece la dirección hacia el objetivo, α.
  - **Ejemplo** para el promedio incremental.
    - Para obtener una nueva estimación para un promedio, usamos el promedio antiguo y un tamaño de paso.
    - El parámetro $\alpha = TamañodelPaso = 1/n$ cambia de paso de tiempo a paso de tiempo.
    - Error de la estimación: $(Objetivo - ViejoEstimado)$
    - $\alpha_n = 1/n$ establece la dirección hacia el objetivo.

Comparación entre Técnicas de Aprendizaje por Refuerzo.

Técnica	Euación de Bellman y su Fórmula de Aproximación
Monte Carlo	$v_\pi(s) = \mathbb{E}[G_t \mid S_t = s]$ $V(S_t) \leftarrow V(S_t) + \alpha[G_t - V(S_t)]$
TD(0)	$v_\pi(s) = \mathbb{E}[R_{t+1} + \gamma v_\pi(S_{t+1}) \mid S_t = s]$ $V(S_t) \leftarrow V(S_t) + \alpha[R_{t+1} + \gamma V(S_{t+1}) - V(S_t)]$
SARSA	$q_\pi(s,a) = \mathbb{E}[R_{t+1} + \gamma q_\pi(S_{t+1}, A_{t+1}) \mid S_t = s, A_t = a]$ $Q(S_t, A_t) \leftarrow Q(S_t, A_t) + \alpha[R_{t+1} + \gamma Q(S_{t+1}, A_{t+1}) - Q(S_t, A_t)]$
Q-Learning	$q_*(s,a) = \mathbb{E}[R_{t+1} + \gamma \operatorname{máx}_{a'} q_*(S_{t+1}, a') \mid S_t = s, A_t = a]$ $Q(S_t, A_t) \leftarrow Q(S_t, A_t) + \alpha[R_{t+1} + \gamma \operatorname{máx}_a Q(S_{t+1}, a) - Q(S_t, A_t)]$
Expected SARSA	$q_\pi(s,a) = \mathbb{E}[R_{t+1} + \gamma \sum_{a'} \pi(a' \mid S_{t+1}) q_\pi(S_{t+1}, a')]$ $Q(S_t, A_t) \leftarrow Q(S_t, A_t) + \alpha\,[R_{t+1} + \gamma \sum_{a'} \pi(a' \mid S_{t+1}) Q(S_{t+1}, a') - Q(S_t, A_t)]$

Desarrollo

1. Introducción

2. Cálculo Incremental

3. Predicción
   3.1. Diferencias Temporales a un Paso: TD(0)
   3.2. Optimalidad de TD(0) - Por lotes

4. Control
   4.1. SARSA
   4.2. Q-Learning
   4.3. Sesgo de Maximización
      Regularización de Valores
      Promedio de Valores
         (Expected SARSA y Aprendizaje doble)

5. Referencias

Profundizamos en ...

1. Introducción

2. Cálculo Incremental

3. Predicción
   3.1. Diferencias Temporales a un Paso: TD(0)
   3.2. Optimalidad de TD(0) - Por lotes

4. Control
   4.1. SARSA
   4.2. Q-Learning
   4.3. Sesgo de Maximización
        Regularización de Valores
        Promedio de Valores
            (Expected SARSA y Aprendizaje doble)

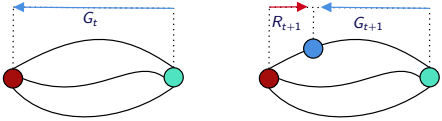
5. Referencias

Monte Carlo α-constante y TD(0)

- En **Monte Carlo** nuestro objetivo es calcular: $v_\pi(s) \doteq \mathbb{E}_\pi[G_t|S_t = s]$
- Pero al no tener modelo, **calculamos un promedio**: $v_\pi(s) \doteq \mathbb{E}_\pi[G_t|S_t = s] \approx \frac{1}{N}\sum_{i=1}^{N} G_t^{(i)}$
- Si aplicamos el **cálculo incremental** (para todos los estados - aplicado en predicción MC):
  - $N(S_t) \leftarrow N(S_t) + 1$
  - $V(S_t) \leftarrow V(S_t) + \frac{1}{N(S_t)}\Big(G_t - V(S_t)\Big)$
- **MC con α constante**: $V(S_t) \leftarrow V(S_t) + \alpha\,(G_t - V(S_t))$
- Sabemos que:

$$v_\pi(s) \doteq \mathbb{E}_\pi[G_t|S_t = s] = \mathbb{E}_\pi[R_{t+1} + \gamma G_{t+1}|S_t = s] = \mathbb{E}_\pi[R_{t+1} + \gamma v_\pi(S_{t+1})|S_t = s]$$

- **TD(0)**: $V(S_t) \leftarrow V(S_t) + \alpha\,(R_{t+1} + \gamma V(S_{t+1}) - V(S_t))$ **Error DT**: $\delta_t = R_{t+1} + \gamma V(S_{t+1}) - V(S_t)$
- Las diferencias **de forma gráfica**:



Algorithm 1 TD(0) tabular para estimar v_π

Tabular TD(0) for estimating v_π

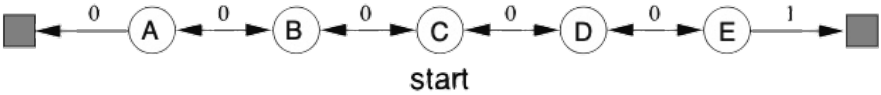
Input: the policy π to be evaluated
Algorithm parameter: step size $\alpha \in (0, 1]$
Initialize $V(s)$, for all $s \in \mathcal{S}^+$, arbitrarily except that $V(terminal) = 0$

Loop for each episode:
  Initialize S
  Loop for each step of episode:
    $A \leftarrow$ action given by π for S
    Take action A, observe R, S'
    $V(S) \leftarrow V(S) + \alpha\big[R + \gamma V(S') - V(S)\big]$
    $S \leftarrow S'$
  until S is terminal

Ventajas de los Métodos de Predicción TD

- Los **métodos TD** tienen una ventaja **sobre los métodos de Programación Dinámica** (DP) en que **no requieren un modelo del entorno**, de sus recompensas y distribuciones de probabilidad del siguiente estado.
- Los **métodos TD** tienen una ventaja **sobre los métodos de Monte Carlo** (MC) en que están **naturalmente implementados en línea** (on-line), de forma completamente incremental.
- **Los métodos TD convergen**. Para cualquier política fija π, con $TD(0)$ se garantiza que $\lim_{t\to\infty} V_t(s) = v_\pi(s)$ con **probabilidad 1** si el parámetro de tamaño de paso **disminuye** de acuerdo con las **condiciones de aproximación estocástica** (Condiciones de Robbins-Monro) para la tasa de aprendizaje α_n:
$$\sum_{n=1}^{\infty} \alpha_n(a) = \infty \quad \text{y} \quad \sum_{n=1}^{\infty} \alpha_n^2(a) < \infty.$$
- **En la práctica**, los métodos TD generalmente han mostrado una **velocidad de convergencia más rápida que los métodos Monte Carlo constante-α** en tareas estocásticas; pero no está demostrado.

Ejemplo: Paseo Aleatorio



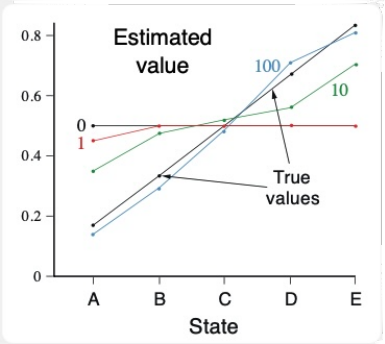
Ejemplo de Episodio: $C, 0, B, 0, C, 0, D, 0, E, 1$

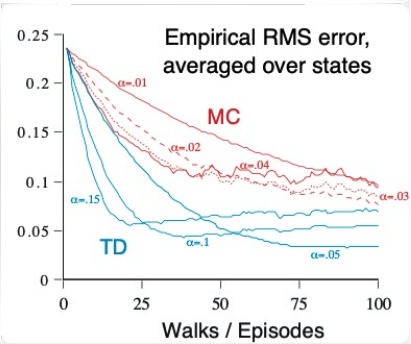
$v(A) = \frac{1}{6}$, $v(B) = \frac{2}{6}$, $v(C) = \frac{3}{6}$,
$v(D) = \frac{4}{6}$, $v(E) = \frac{5}{6}$.

Ecuaciones de Bellman:

$v(s) = \sum_{s'} P(s' \mid s)[R(s, s') + \gamma v(s')]$

$v(A) = 0{,}5 \cdot v(B)$, $v(B) = 0{,}5 \cdot v(A) + 0{,}5 \cdot v(C)$,
$v(C) = 0{,}5 \cdot v(B) + 0{,}5 \cdot v(D)$, $v(D) = 0{,}5 \cdot v(C) + 0{,}5 \cdot v(E)$,
$v(E) = 0{,}5 \cdot v(D) + 0{,}5$.





Profundizamos en ...

1. Introducción

2. Cálculo Incremental

3. Predicción
   3.1. Diferencias Temporales a un Paso: TD(0)
   3.2. Optimalidad de TD(0) – Por lotes

4. Control
   4.1. SARSA
   4.2. Q-Learning
   4.3. Sesgo de Maximización
      Regularización de Valores
      Promedio de Valores
         (Expected SARSA y Aprendizaje doble)

5. Referencias

Actualización por lotes (Batch updating)

- Observa las expresiones:

$$V(S_t) \leftarrow V(S_t) + \alpha\,(G_t - V(S_t)) \qquad\qquad V(S_t) \leftarrow V(S_t) + \alpha\,(R_{t+1} + \gamma V(S_{t+1}) - V(S_t))$$

- ¿Por qué no usar un lote (conjunto de muestras) en vez de solo una muestra?
- Batch updating (ver Lange et al., 2012 para más técnicas):
- Para **TD(0) con α constante**

Algorithm 2 Diferencias Temporales por Lotes con Tasa de Aprendizaje Constante

Entrada: α (tasa de aprendizaje constante), γ (factor de descuento), π (política)

```
Inicializar V(s) ← 0 para todo s ∈ S
while no convergencia do
    Recopilar un lote de transiciones D = {(s_t, A_t, R_{t+1}, s_{t+1})} a partir de episodios aleatorios.
    for cada transición (s_t, A_t, R_{t+1}, s_{t+1}) en D do
        Calcular el error de TD: δ(s_t) ← R_{t+1} + γV(s_{t+1}) − V(s_t)
        Actualizar la función de valor: V(s_t) ← V(s_t) + αδ(s_t)
    end for   Repetir el for hasta que "converga" si solo hay un lote
    Verificar convergencia de V(s) (p.e. máx_s |V(s) − V_anterior(s)| < ε entre 2 lotes consecutivos)
end while
```

- **Usa TD(0) sin lotes** si…

  Necesitas respuestas rápidas y en línea. Estás trabajando con sistemas en tiempo real. Hay flujo continuo de datos. El entorno cambia constantemente.

- **Usa TD(0) por Lotes** si…

  Quieres estabilidad en el aprendizaje. Tienes un conjunto limitado de datos (aprendizaje fuera de línea). Quieres mejorar la eficiencia computacional en modelos grandes.

- Para **MC α-constant**, actualizar $V(s)$ como el **promedio de todos los retornos observados** en el lote:

$$V(s) \leftarrow V(s) + \alpha \cdot \left(\frac{1}{n(s)} \sum G(s) - V(s) \right)$$

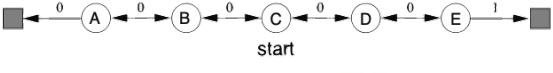
  - $n(s)$ es el número de veces que se ha visitado s en el lote.

Optimalidad de TD(0)

Actualización por lotes (Batch updating):

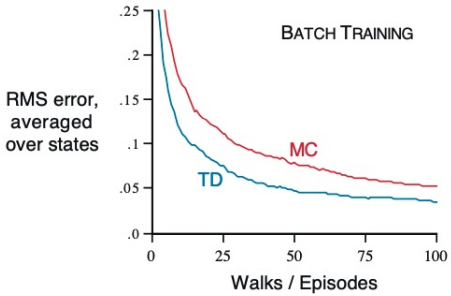
- **TD(0) converge determinísticamente** a una única solución independiente del parámetro de tamaño de paso, α, siempre que α sea suficientemente pequeño.
- El método de **Monte Carlo** constante-α también **converge determinísticamente** bajo las mismas condiciones, pero a una solución diferente.

Ejemplo: *Paseo Aleatorio*



;

$$\text{RMS error} = \sqrt{\frac{1}{|S|}\sum_{s\in S}(V(s) - v_\pi(s))^2}$$



Optimalidad de TD(0)
Adivina adivinanza ....

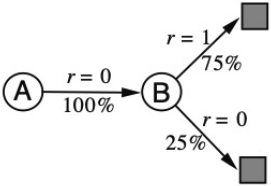
- Considera un **Proceso de Recompensa de Markov** desconocido (no tiene acciones).
- Supón que **observas los siguientes ocho episodios**:

A, 0, B, 0	B, 1
B, 1	B, 1
B, 1	B, 1
B, 1	B, 0

- ¿Cuánto debería ser el **valor esperado del retorno** para cada estado?

MC	TD(0)
v(B)=6/8=3/4	v(B) = 3/4
v(A)=0	v(A) = 3/4

- ¿Por qué?



;

§Optimalidad de TD(0)
Resultados Finales

- Los métodos **Monte Carlo por lotes** siempre encuentran las estimaciones que **minimizan** el **error cuadrático medio en el conjunto de entrenamiento**.
- **Batch TD(0)** siempre encuentra las estimaciones que serían exactamente correctas para el **modelo de máxima verosimilitud del proceso de Markov**:
  - Una estimación de máxima verosimilitud de un parámetro es el **valor del parámetro cuya probabilidad de generar los datos es la mayor**. $\hat{\theta} = \arg\max_\theta P(\text{Datos} \mid \theta)$
  - La **probabilidad estimada de transición de i a j** es el promedio de transiciones observadas de i a j,
  $$P(i \to j) = \frac{\text{Número de transiciones observadas de } i \text{ a } j}{\text{Número total de transiciones desde } i}$$
  - La **recompensa esperada** asociada es el promedio de las recompensas observadas en esas transiciones.
  $$R(i \to j) = \frac{\text{Suma de recompensas observadas en las transiciones de } i \text{ a } j}{\text{Número de transiciones observadas de } i \text{ a } j}$$
- Dado este modelo, podemos calcular la **estimación de la función de valor que sería exactamente correcta si el modelo fuera exactamente correcto**.
$$V(s) = \sum_{j \in S} P(s \to j)\left[R(s \to j) + \gamma V(j)\right]$$
  - Esto se llama la **estimación de equivalencia a certeza**, porque es equivalente a asumir que la estimación del proceso subyacente se conocía con certeza en lugar de ser aproximada.
- **Batch TD(0) converge a la estimación de equivalencia a certeza**.

Desarrollo

1. Introducción

2. Cálculo Incremental

3. Predicción
   3.1. Diferencias Temporales a un Paso: TD(0)
   3.2. Optimalidad de TD(0) - Por lotes

4. Control
   4.1. SARSA
   4.2. Q-Learning
   4.3. Sesgo de Maximización
        Regularización de Valores
        Promedio de Valores
            (Expected SARSA y Aprendizaje doble)

5. Referencias

Profundizamos en ...

1. Introducción

2. Cálculo Incremental

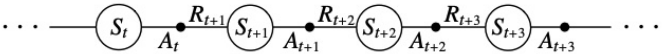
3. Predicción
   3.1. Diferencias Temporales a un Paso: TD(0)
   3.2. Optimalidad de TD(0) - Por lotes

4. Control
   4.1. SARSA
   4.2. Q-Learning
   4.3. Sesgo de Maximización
        Regularización de Valores
        Promedio de Valores
            (Expected SARSA y Aprendizaje doble)

5. Referencias

Control TD On-Policy: SARSA
$(S_t, A_t, R_{t+1}, S_{t+1}, A_{t+1})$

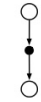
Parte de un episodio



- Seguimos el enfoque de **Iteración Generalizada de Políticas (GPI)** utilizando métodos de diferencias temporales (TD) para la parte de evaluación de la política.
- **Método On-Policy:**
  - Debemos estimar $q_\pi(s, a)$ para la política de comportamiento actual π y para todos los estados s y acciones a.
  - Consideramos las transiciones de pares estado-acción a pares estado-acción, y aprendemos los valores de dichos pares.
- La **fórmula de actualización de Sarsa** es:

$$Q(S_t, A_t) \leftarrow Q(S_t, A_t) + \alpha\left[R_{t+1} + \gamma Q(S_{t+1}, A_{t+1}) - Q(S_t, A_t)\right]$$

  - Esta actualización se realiza **después de cada transición desde un estado S_t que no sea terminal**.
  - Si S_{t+1} es un estado terminal, entonces $Q(S_{t+1}, A_{t+1})$ se define como cero.



TD(0)

Sarsa

Alagoritmo SARSA

Algorithm 3 Sarsa (Control TD sobre la política) para estimar $Q \approx q_*$

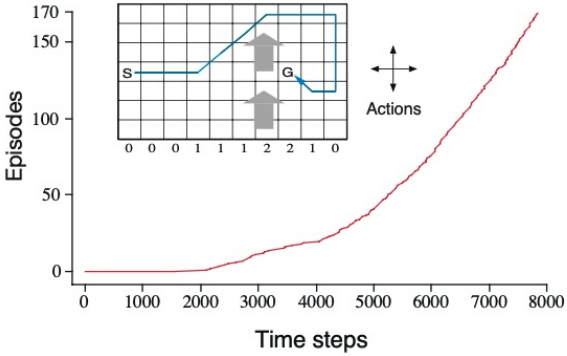
Sarsa (on-policy TD control) for estimating $Q \approx q_*$

Algorithm parameters: step size $\alpha \in (0, 1]$, small $\varepsilon > 0$
Initialize $Q(s, a)$, for all $s \in \mathcal{S}^+, a \in \mathcal{A}(s)$, arbitrarily except that $Q(terminal, \cdot) = 0$

Loop for each episode:
  Initialize S — $P(A = \text{arg máx}_a\, Q(s, a)) = 1 - \epsilon$
  Choose A from S using policy derived from Q (e.g., ε-greedy)
  Loop for each step of episode:
    Take action A, observe R, S' — $P(A' = \text{arg máx}_a\, Q(s', a)) = 1 - \epsilon$
    Choose A' from S' using policy derived from Q (e.g., ε-greedy)
    $Q(S, A) \leftarrow Q(S, A) + \alpha\big[R + \gamma Q(S', A') - Q(S, A)\big]$
    $S \leftarrow S'$; $A \leftarrow A'$; — $P(\pi(s) = \text{arg máx}_a\, Q(s, a)) = 1 - \epsilon$
  until S is terminal

- **SARSA converge** con probabilidad 1 a una política óptima y una función de valor-acción, **siempre que**
  - Todos los pares estado-acción se visiten un **número infinito** de veces
  - Cumpla las **condiciones de aproximación estocástica** sobre los tamaños de paso (pág. 12)
  - La política **converja** en el límite hacia la política codiciosa.

Windy Grid World
Algoritmo SARSA



- Eje X: Número total de pasos de tiempo ejecutados (hasta 8000 pasos).
- Eje Y: Número de episodios completados por el agente.
  Cada episodio comienza en el estado inicial (**S**) y termina al alcanzar el estado objetivo (**G**).

- Agente se mueve en 4 direcciones.
- Hay un viento cruzado.
  Con la intensidad indicada el agente sube.
- $\gamma = 1$, $R(s, s') = -1$, $R(s, t) = 0$
- La solución óptima tiene 15 pasos.
- SARSA
  - Política ϵ-greedy, $\epsilon = 0{,}1$
  - Tamaño de paso cte: $\alpha = 0{,}5$
  - Valores iniciales $Q(s, a) = 0$ para todo s, a

Resultados

- La curva es creciente: a mayor tiempo, más rapido concluye el episodio.
- La política ϵ-greedy:
  - Genera episodios, en promedio, de 17 pasos.
  - Ya era óptima (se muestra la trayectoria);

Profundizamos en ...

1. Introducción

2. Cálculo Incremental

3. Predicción
   3.1. Diferencias Temporales a un Paso: TD(0)
   3.2. Optimalidad de TD(0) - Por lotes

4. Control
   4.1. SARSA
   4.2. Q-Learning
   4.3. Sesgo de Maximización
      Regularización de Valores
      Promedio de Valores
         (Expected SARSA y Aprendizaje doble)

5. Referencias

Control TD Off-Policy: Q-Learning
(Watkins, 1989)

- **Off-Policy.**
  - **Política de comportamiento:** Cualquiera que explore.
  - **Política objetivo:** $\pi_*(s) = \arg\max_a q_*(s,a)$, $q_*(s,a) = \mathbb{E}[R_{t+1} + \gamma \max_{a'} q_*(S_{t+1}, a') | S_t = s, A_t = a]$
- Recordemos la **Ecuación Óptima** de Bellman:

$$q_*(s,a) = q_{\pi_*}(s,a) = \mathbb{E}[R_{t+1} + \gamma \max_{a'} q_*(S_{t+1}, a') | S_t = s, A_t = a]$$

- La función de valor-acción aprendida, Q, **aproxima directamente** q_*, **independientemente** de la **política**.

$$Q(S_t, A_t) \leftarrow Q(S_t, A_t) + \alpha \left[R_{t+1} + \gamma \max_a Q(S_{t+1}, a) - Q(S_t, A_t) \right]$$

- Todo lo que se requiere **para la convergencia** a q_*, con probabilidad 1, es que todos los pares continúen siendo actualizados y las condiciones de tamaño de paso (pág. 12).
- La **política** *todavía tiene un efecto* en el sentido de que determina qué pares estado-acción son visitados y actualizados; pero no determina la exploración.

Alagoritmo Q-Learning

Algorithm 4 Q-Learning (Control DT off-policy) para estimar $\pi \approx \pi_*$

Q-learning (off-policy TD control) for estimating $\pi \approx \pi_*$

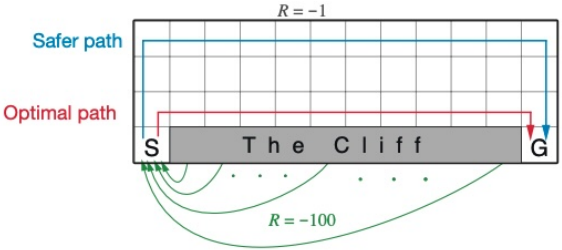
Algorithm parameters: step size $\alpha \in (0, 1]$, small $\varepsilon > 0$
Initialize $Q(s, a)$, for all $s \in \mathcal{S}^+, a \in \mathcal{A}(s)$, arbitrarily except that $Q(terminal, \cdot) = 0$

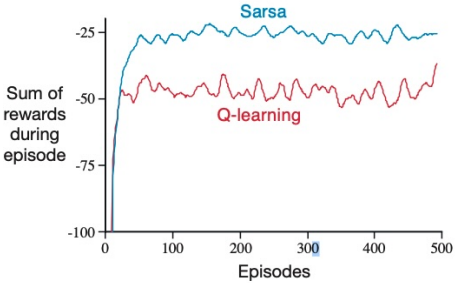
Loop for each episode:
  Initialize S
  Loop for each step of episode:
    Choose A from S using policy derived from Q (e.g., ε-greedy)
    Take action A, observe R, S'
    $Q(S, A) \leftarrow Q(S, A) + \alpha\big[R + \gamma \max_a Q(S', a) - Q(S, A)\big]$
    $S \leftarrow S'$
  until S is terminal

$b(A = \text{arg máx}_a\, Q(s, a)) = 1 - \epsilon$

$\pi(s) = \text{arg máx}_a\, Q(s, a)$

Ejemplo: Caminando por el Acantilado
Comparación entre Sarsa y Q-learning





- Mundo grid donde el agente se mueve en las 4 direcciones. Debe llegar de **S** a **G**.
- Si cae por el acantilado vuelve a **S**.
- Recompensa es -1 en todas las transiciones, excepto si cae en el acantilado que será de -100.
- Se aplica SARSA y Q-learning con políticas ϵ-greedy ($\epsilon = 0{,}1$)

Profundizamos en ...

1. Introducción

2. Cálculo Incremental

3. Predicción
   3.1. Diferencias Temporales a un Paso: TD(0)
   3.2. Optimalidad de TD(0) - Por lotes

4. Control
   4.1. SARSA
   4.2. Q-Learning
   4.3. Sesgo de Maximización
      Regularización de Valores
      Promedio de Valores
         (Expected SARSA y Aprendizaje doble)

5. Referencias

Sesgo de Maximización

- En **Q-learning** y (a menudo) **Sarsa**, consideran el **máximo sobre valores** que pueden estar sobreestimados, **lo que puede llevar a un sesgo positivo significativo.**
  - En **Q-learning**, la política objetivo es la política codiciosa dada por los valores de acción actuales, lo cual se define con una operación **máx**.
  - En **Sarsa**, la política suele ser ϵ**-codiciosa**, lo que también implica una operación de maximización.
- **Sesgo de Maximización (Maximization Bias).**
  - Considere un único estado s con muchas acciones a cuyos valores verdaderos, $q(s, a)$, son **todos cero**.
    - P.e. $q(s, a_1) = 0, \quad q(s, a_2) = 0, \quad q(s, a_3) = 0.$
  - El **máximo de los valores verdaderos es cero**, $max_a q(s, a) = 0$.
  - Los valores estimados, $Q(s, a)$, están distribuidos alrededor de cero.
    - P.e. $Q(s, a_1) = 0{,}1, \quad Q(s, a_2) = -0{,}05, \quad Q(s, a_3) = 0{,}2. \Rightarrow \text{máx}\, Q(s, a) = 0{,}2 \quad (\text{acción } a_3)$
  - Pero si el **máximo de las estimaciones** $Q(s, a)$ **es positivo**, se crea un **sesgo positivo**.
    - P.e. $\text{máx}\, Q(s, a) = 0{,}2 \quad (\text{acción } a_3)$
  - **Sesgo Positivo:** $\mathbb{E}[\text{máx}_a\, Q(s, a)] > \text{máx}_a\, \mathbb{E}[Q(s, a)]$
- **Soluciones:**
  - **Exploración continua (ϵ-greedy):** Mantiene la exploración incluso en etapas avanzadas para evitar quedarse atrapado en valores sobreestimados.
  - **Regularización de valores:** Reduce los efectos de valores extremos aplicando restricciones o penalizaciones.
  - **Promedio de valores:** Usa promedios ponderados en lugar de seleccionar el máximo.
  - **Aprendizaje Doble (Double Learning):** Divide el aprendizaje en dos conjuntos de valores Q_1 y Q_2.

Con más detalle ..

1. Introducción

2. Cálculo Incremental

3. Predicción
   3.1. Diferencias Temporales a un Paso: TD(0)
   3.2. Optimalidad de TD(0) - Por lotes

4. Control
   4.1. SARSA
   4.2. Q-Learning
   4.3. Sesgo de Maximización
   Regularización de Valores
   Promedio de Valores
   (Expected SARSA y Aprendizaje doble)

5. Referencias

Regularización de Valores
Ejemplos de regularización:

- **Penalización L1 (Sparsity):** - Fomenta soluciones más compactas y esparsas.
$$Q(s,a) \leftarrow Q(s,a) + \alpha[R + \gamma \max_a Q(s',a) - Q(s,a)] - \lambda|Q(s,a)|$$
- **Penalización L2 (Decay):** - Reduce el crecimiento excesivo de los valores Q.
$$Q(s,a) \leftarrow Q(s,a) + \alpha[R + \gamma \max_a Q(s',a) - Q(s,a)] - \lambda Q(s,a)$$
- **Reducción por temperatura:** - Controlar suavemente la exploración y explotación
$$Q(s,a) \leftarrow Q(s,a) + \alpha\left[R + \gamma \frac{\sum_a Q(s',a)e^{Q(s',a)/T}}{\sum_a e^{Q(s',a)/T}} - Q(s,a)\right]$$
- **Clipping de valores:** - Restringe valores extremos para evitar explosiones numéricas.
$$Q(s,a) \leftarrow \min\left(Q_{max}, \max\left(Q(s,a), Q_{min}\right)\right)$$

  - **Límites teóricos:** $Q_{max} = \frac{R_{max}}{1-\gamma}, \quad Q_{min} = \frac{R_{min}}{1-\gamma}$
  - **Recompensas observadas:** $Q_{max} = R_{obs} + \delta, \quad Q_{min} = R_{obs} - \delta$
  - **Basados en la escala de los valores** Q: $Q_{max} = \mu + k\sigma, \quad Q_{min} = \mu - k\sigma$
  - **Ajuste dinámico:** $Q_{max}(t+1) = \max(Q_{max}(t), Q(s,a) + \delta)$ $Q_{min}(t+1) = \min(Q_{min}(t), Q(s,a) - \delta)$

Con más detalle ..

1. Introducción

2. Cálculo Incremental

3. Predicción
   3.1. Diferencias Temporales a un Paso: TD(0)
   3.2. Optimalidad de TD(0) - Por lotes

4. Control
   4.1. SARSA
   4.2. Q-Learning
   4.3. Sesgo de Maximización
      Regularización de Valores
      Promedio de Valores
         (Expected SARSA y Aprendizaje doble)

5. Referencias

Un Resumen de Técnicas Basadas en Promedios de Valores

- **Expected SARSA:** – Promedia los valores ponderados por una política exploratoria (π).

$$Q(S_t, A_t) \leftarrow Q(S_t, A_t) + \alpha \left[R_{t+1} + \gamma \sum_{a'} \pi(a' \mid S_{t+1}) Q(S_{t+1}, a') - Q(S_t, A_t) \right]$$

- **Double Q-Learning con Promedio:** – Usa dos estimadores (Q_1 y Q_2) y promedia sus valores.

$$Q_1(s, a) \leftarrow Q_1(s, a) + \alpha \left[R + \gamma Q_2(s', \arg\max_a Q_1(s', a)) - Q_1(s, a) \right] \qquad Q(s, a) = \frac{Q_1(s, a) + Q_2(s, a)}{2}$$

- **Soft Q-Learning:** – Sustituye el máximo por un promedio suavizado mediante una distribución Boltzmann.

$$Q(s, a) \leftarrow Q(s, a) + \alpha \left[R + \gamma \frac{\sum_{a'} Q(s', a') e^{Q(s', a')/T}}{\sum_b e^{Q(s', b)/T}} - Q(s, a) \right]$$

- **Soft Actor-Critic (SAC):** – Añade un término de entropía para equilibrar exploración y explotación.

$$Q(s, a) \leftarrow Q(s, a) + \alpha \left[R + \gamma \mathbb{E}_{a'}[Q(s', a') - \tau \log \pi(a' \mid s')] - Q(s, a) \right]$$

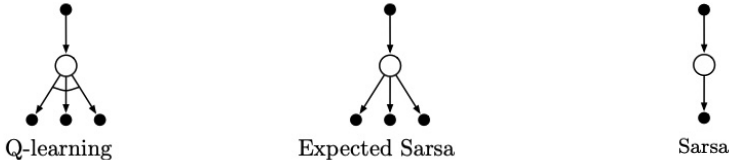
- **Expected Value SARSA (EV-SARSA):** – Promedia valores Q sobre múltiples simulaciones desde un estado.

$$Q(s, a) \leftarrow Q(s, a) + \alpha \left[R + \gamma \frac{1}{N} \sum_{i=1}^{N} Q(s', a_i) - Q(s, a) \right]$$

Expected SARSA

- **Expected Sarsa** es como Q-Learning pero utiliza el valor esperado, teniendo en cuenta qué tan probable es cada acción bajo la política actual.

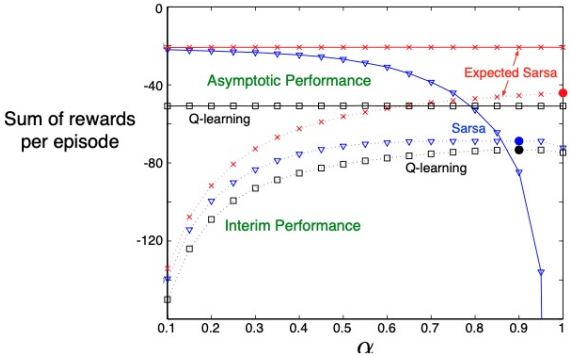
$$
\begin{aligned}
Q(S_t, A_t) &\leftarrow Q(S_t, A_t) + \alpha\left[R_{t+1} + \gamma \mathbb{E}_\pi[Q(S_{t+1}, A_{t+1}) | S_{t+1}] - Q(S_t, A_t)\right] \\
&= Q(S_t, A_t) + \alpha\left[R_{t+1} + \gamma \sum_a \pi(a|S_{t+1}) Q(S_{t+1}, a) - Q(S_t, A_t)\right],
\end{aligned}
$$



- **Expected SARSA** **es una generalización** de Q-Learning; pero suele usare un política soft.
- Dado el siguiente estado, S_{t+1}, este algoritmo **se mueve de manera determinista** en la misma dirección en que **SARSA se mueve en expectativa**.
- **Expected SARSA** es más complejo computacionalmente que SARSA, pero **elimina la varianza** debido a la selección aleatoria de A_{t+1}.

Ejemplo: Caminando por el Acantilado
Comparación entre SARSA, Q-Learning y Expected SARSA

- Todos los algoritmos utilizaron una política ϵ-codiciosa con $\epsilon = 0{,}1$.
- Una ejecución consiste en un conjunto de episodios consecutivos.
- Estos datos son promedios calculados a partir de más de **50,000 ejecuciones** para los casos intermedios y **10 ejecuciones** para los casos asintóticos, respectivamente.
- El número de **episodios por ejecución** depende del tipo de evaluación que se esté realizando:
  - Para la evaluación *intermedia*, cada ejecución tiene 100 episodios.
  - Para la evaluación *asintótica*, cada ejecución tiene 100,000 episodios.
- Los círculos sólidos marcan el mejor rendimiento **intermedio** de cada método.



$$\bar{G} = \frac{1}{50,000} \sum_{i=1}^{50,000} \left[\frac{1}{100} \sum_{j=1}^{100} G_{ij} \right].$$

$$G_{i,j} = R_1 + R_2 + R_3 + \ldots + R_T$$

☞ Expected Sarsa puede dominar completamente a dos los algoritmo de control DT más conocidos, si no fuera por su coste computacional.

Aprendizaje Doble

- **Q-Learning:** $Q(S_t, A_t) \leftarrow Q(S_t, A_t) + \alpha\left[R_{t+1} + \gamma \max_a Q(S_{t+1}, a) - Q(S_t, A_t)\right]$
- El problema del **sesgo de maximización** surge porque usamos las **mismas muestras** tanto para
  - **Determinar la acción que maximiza** ($A^* = \arg\max Q(a)$).
  - **Estimar el valor** de esa acción ($Q(A^*) = Q(\arg\max Q(a))$).
- **Aprendizaje Doble:**
  - Divide las **muestras en dos conjuntos** para aprender **dos estimaciones lo suficientemente independientes**, $Q_1(a)$ y $Q_2(a)$, del valor verdadero $q(a)$ para cada $a \in \mathcal{A}$.
  - Una estimación se usa para determinar la **acción que maximiza**: $A^* = \arg\max Q_1(a)$
  - La otra estimación proporciona el **valor de esa acción**: $Q_2(A^*) = Q_2\left(\arg\max Q_1(a)\right)$
  - Se repite el proceso intercambiando los roles de las dos estimaciones para obtener un **segundo valor no sesgado**: $Q_1\left(\arg\max Q_2(a)\right)$
- **Observaciones sobre el Aprendizaje Doble**
  - Al dividir los valores Q en dos estimaciones Q_1 y Q_2, permite actualizaciones más **estables** y reduce errores en la optimización – el **sesgo positivo** causado por el uso del mismo conjunto de datos para seleccionar acciones y estimar sus valores.
  - Aprendemos dos estimaciones, pero **solo una estimación se actualiza en cada paso**.
  - El aprendizaje doble **duplica los requisitos de memoria**, pero **no incrementa la cantidad de cómputo por paso**.

Double Q-learning para estimar $Q_1 \sim Q_2 \sim q_0$

Algorithm 5 Double Q-learning para estimar $Q_1 \sim Q_2 \sim q_0$

Double Q-learning, for estimating $Q_1 \approx Q_2 \approx q_*$

Algorithm parameters: step size $\alpha \in (0, 1]$, small $\varepsilon > 0$
Initialize $Q_1(s, a)$ and $Q_2(s, a)$, for all $s \in \mathcal{S}^+, a \in \mathcal{A}(s)$, such that $Q(terminal, \cdot) = 0$

Loop for each episode:
  Initialize S
  Loop for each step of episode:
    Choose A from S using the policy ε-greedy in $Q_1 + Q_2$
    Take action A, observe R, S'
    With 0.5 probabilility:

$$Q_1(S, A) \leftarrow Q_1(S, A) + \alpha\Big(R + \gamma Q_2\big(S', \operatorname{argmax}_a Q_1(S', a)\big) - Q_1(S, A)\Big)$$

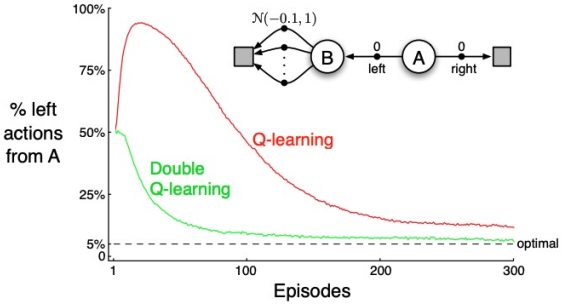
    else:

$$Q_2(S, A) \leftarrow Q_2(S, A) + \alpha\Big(R + \gamma Q_1\big(S', \operatorname{argmax}_a Q_2(S', a)\big) - Q_2(S, A)\Big)$$

    $S \leftarrow S'$
  until S is terminal

Comparación entre Q-learning y Double Q-learning

- $Q(s,a) \leftarrow Q(s,a) + \alpha\left[R + \gamma \max_{a'} Q(s',a') - Q(s,a)\right]$
- $Q_i(s,a) \leftarrow Q_i(s,a) + \alpha\left[R + \gamma Q_j(s', \arg\max_{a'} Q_i(s',a')) - Q_i(s,a)\right]$



- El MDP tiene dos estados no terminales, A y B.
- Si está en B tiene muchas acciones al nodo terminal $q_\pi(B,a) = \mathbb{E}[G] = -0{,}1$
- Siempre comienzan en A, con recompensas laterales nulas. $q_\pi(A, izda) = -0{,}1\gamma$, $q_\pi(A, dch) = 0$
- La política óptima para A es ir a la derecha.
- Política $\epsilon = 0{,}1$-greedy
- $\gamma = 0{,}1$, y $\alpha = 1$
- 10,000 ejecuciones de 300 episodios.

Desarrollo

1. Introducción

2. Cálculo Incremental

3. Predicción
   3.1. Diferencias Temporales a un Paso: TD(0)
   3.2. Optimalidad de TD(0) - Por lotes

4. Control
   4.1. SARSA
   4.2. Q-Learning
   4.3. Sesgo de Maximización
   Regularización de Valores
   Promedio de Valores
   (Expected SARSA y Aprendizaje doble)

5. Referencias

Referencias

- Lange, S., Gabel, T., & Riedmiller, M. (2012). Batch Reinforcement Learning. En *Reinforcement Learning* (pp. 45–73). Springer. https://doi.org/10.1007/978-3-642-27645-3_2
- Silver, D. (2015). Lectures on Reinforcement Learning. https://www.davidsilver.uk/teaching/
- Sutton, R., & Barto, A. (2018). *Reinforcement Learning, second edition: An Introduction*. MIT Press. https://web.stanford.edu/class/psych209/Readings/SuttonBartoIPRLBook2ndEd.pdf
- Watkins, C. J. C. H. (1989). *Learning from Delayed Rewards* [Tesis doctoral, University of Cambridge]. https://ethos.bl.uk/OrderDetails.do?uin=uk.bl.ethos.330022